

D3M ASSIGNMENT – 1

Name : Sanjai Balaji

Student Number : 40478904

Module : Data driven decision making

University : Queen's University Belfast

Task 1 – Semiannual Capacity Planning for Endoscopy Unit

The hospital's endoscopy unit must allocate ten procedure rooms across a planning horizon of 26 weeks. Each room can be configured for diagnostic procedures, therapeutic procedures, or remain unused during a given week. The hospital must satisfy the weekly demand for diagnostic and therapeutic procedures while respecting room capacities and clinician availability.

Rooms differ in size and therefore have different setup costs, operating costs, and procedure capacities. The objective is to determine the optimal weekly room configurations and procedure allocations that minimise the total operating and setup cost over the planning horizon.

To solve this planning problem, it is formulated as a Mixed Integer Linear Programming (MILP) model.

Parameters Explanation

- **$x_diag[r,w]$ / $x_thera[r,w]$** : Binary variables indicating whether room r is configured for diagnostic or therapeutic procedures in week w . Determines which rooms are active for which procedure.
- **$d[r,w]$ / $t[r,w]$** : Number of diagnostic or therapeutic procedure hours scheduled in room r during week w . Ensures demand is met within room capacity.
- **$s[r,w]$** : Binary variable indicating if a room is newly set up in week w . Triggers the setup cost.
- **$DiagCap_r$ / $TheraCap_r$** : Maximum weekly procedure hours a room can handle depending on its type. Ensures the schedule does not exceed physical limits.
- **S_r / A_r** : Setup cost and operating cost per hour for room r . Used in the objective function to minimise total cost.
- **C_w** : Available clinician hours in week w . Limits total procedures scheduled to what staff can handle.

1. Sets and Indices

Let:

$$r = 1, 2, \dots, 10$$

represent the procedure rooms.

$$w = 1, 2, \dots, 26$$

represent the weeks in the planning horizon.

Rooms are classified into three types:

Room Type Rooms

Small 1–3

Medium 4–7

Large 8–10

2. Decision Variables

The optimisation model uses the following decision variables.

$$x_{r,w}^{diag} = \begin{cases} 1 & \text{if room } r \text{ is configured for diagnostic procedures in week } w \\ 0 & \text{otherwise} \end{cases}$$

$$x_{r,w}^{thera} = \begin{cases} 1 & \text{if room } r \text{ is configured for therapeutic procedures in week } w \\ 0 & \text{otherwise} \end{cases}$$

$$d_{r,w} = \text{number of diagnostic hours scheduled in room } r \text{ during week } w$$

$$t_{r,w} = \text{number of therapeutic hours scheduled in room } r \text{ during week } w$$

$$s_{r,w} = \begin{cases} 1 & \text{if room } r \text{ is newly set up in week } w \\ 0 & \text{otherwise} \end{cases}$$

These variables represent the operational decisions of the hospital, including the configuration of rooms and the allocation of procedure hours.

3. Intermediate Calculations

Each procedure room operates 6 hours per day for 5 days per week, giving:

$$6 \times 5 = 30 \text{ hours per week}$$

However, depending on the room type and equipment availability, rooms can perform more procedures due to scheduling flexibility.

The weekly procedure capacities are defined as:

Room Type Diagnostic Capacity Therapeutic Capacity

Small 60 hours 30 hours

Medium 120 hours 60 hours

Large 180 hours 120 hours

These capacities represent the maximum number of procedure hours that can be scheduled in a room during a week.

4. Objective Function

The objective of the model is to minimise the total cost of operating the endoscopy unit during the 26-week planning period.

$$\min Z = \sum_{r=1}^{10} \sum_{w=1}^{26} S_r s_{r,w} + A_r (d_{r,w} + t_{r,w})$$

where

S_r represents the setup cost for room r , and A_r represents the operating cost per procedure hour.

Room costs are defined as:

Room Type Setup Cost (£) Operating Cost (£/hour)

Small	15000	180
Medium	20000	200
Large	25000	220

The objective function minimises both setup costs and operating costs, ensuring efficient use of hospital resources.

5. Model Constraints

5.1 Room Configuration Constraint

$$x_{r,w}^{diag} + x_{r,w}^{thera} \leq 1 \forall r, w$$

This constraint ensures that each room can be configured for at most one type of procedure per week.

5.2 Demand Satisfaction Constraints

Diagnostic demand:

$$\sum_{r=1}^{10} d_{r,w} \geq P_w^{diag} \forall w$$

Therapeutic demand:

$$\sum_{r=1}^{10} t_{r,w} \geq P_w^{thera} \forall w$$

These constraints ensure that the hospital satisfies the required weekly demand for diagnostic and therapeutic procedures.

5.3 Room Capacity Constraints

Diagnostic procedures:

$$d_{r,w} \leq \text{DiagCap}_r \cdot x_{r,w}^{diag} \forall r, w$$

Therapeutic procedures:

$$t_{r,w} \leq \text{TheraCap}_r \cdot x_{r,w}^{thera} \forall r, w$$

These constraints prevent the number of scheduled procedure hours from exceeding the capacity of each room and ensure that procedures are performed only in appropriately configured rooms.

5.4 Clinician Availability Constraint

$$\sum_{r=1}^{10} (d_{r,w} + t_{r,w}) \leq C_w \forall w$$

This constraint ensures that the total procedure hours scheduled in any week do not exceed the available clinician working hours.

5.5 Setup Logic Constraint

For week 1:

$$s_{r,1} \geq x_{r,1}^{diag} + x_{r,1}^{thera}$$

For weeks 2–26:

$$S_{r,w} \geq x_{r,w}^{diag} + x_{r,w}^{thera} - (x_{r,w-1}^{diag} + x_{r,w-1}^{thera})$$

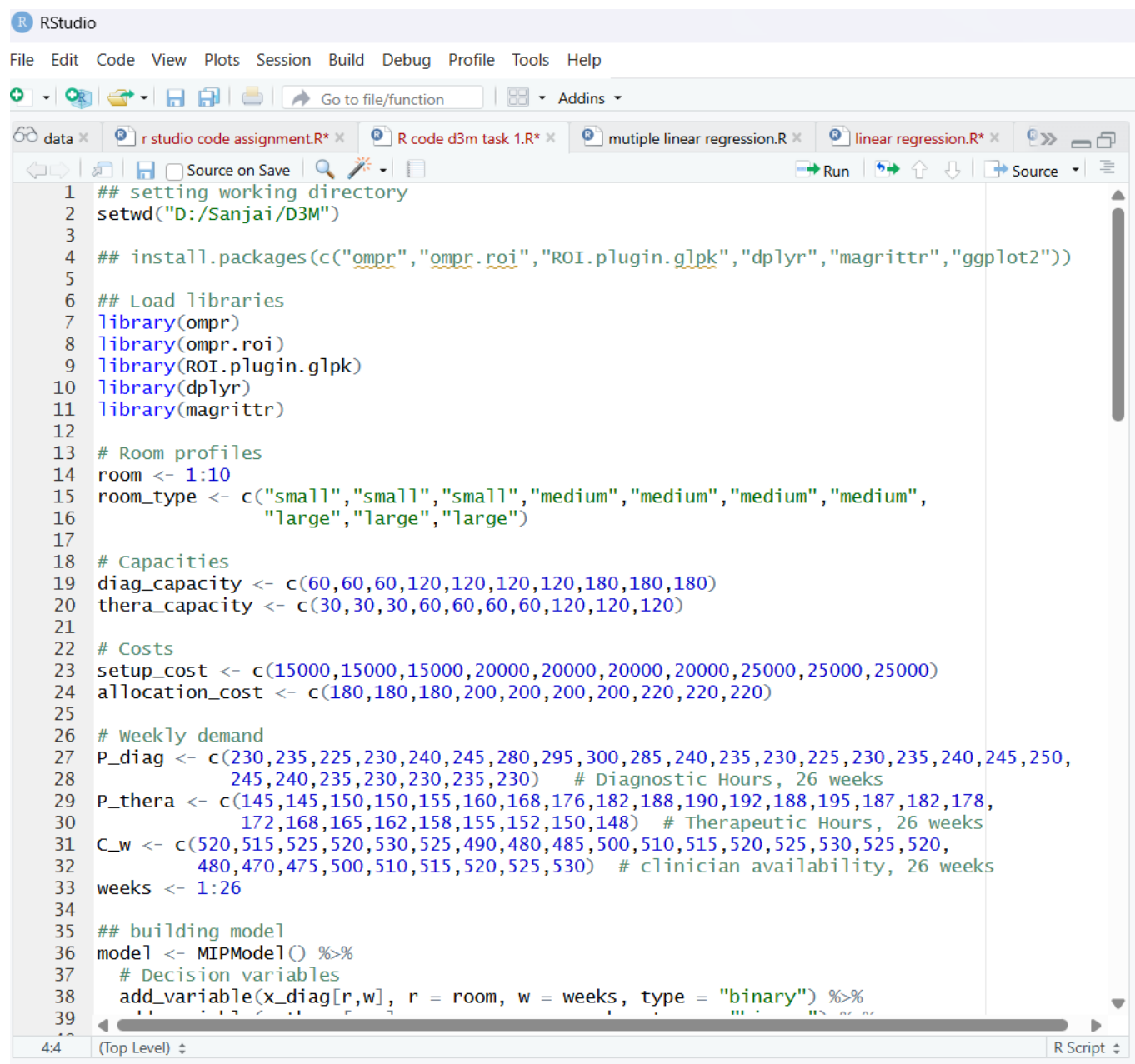
This constraint ensures that a setup cost is incurred whenever a room changes from an inactive state to an active configuration.

6. Implementation in R

The MILP model was implemented in R using the ompr optimisation package. The model was constructed by defining decision variables, the objective function, and the constraints described above. The optimisation model was successfully solved using the GLPK solver, which returned an optimal solution for the 26-week planning horizon.

Short comments were included in the R code to relate each part of the implementation to the corresponding mathematical formulation.

R code screenshots



The screenshot shows the RStudio interface with a script editor containing R code. The code is organized into sections for setting the working directory, installing and loading packages, defining room profiles, capacities, costs, and weekly demand. It also includes the initial setup for building a model using the MIPModel package.

```
1 ## setting working directory
2 setwd("D:/Sanjai/D3M")
3
4 ## install.packages(c("ompr", "ompr.roi", "ROI.plugin.glpk", "dplyr", "magrittr", "ggplot2"))
5
6 ## Load libraries
7 library(ompr)
8 library(ompr.roi)
9 library(ROI.plugin.glpk)
10 library(dplyr)
11 library(magrittr)
12
13 # Room profiles
14 room <- 1:10
15 room_type <- c("small", "small", "small", "medium", "medium", "medium", "medium",
16               "large", "large", "large")
17
18 # Capacities
19 diag_capacity <- c(60, 60, 60, 120, 120, 120, 120, 180, 180, 180)
20 thera_capacity <- c(30, 30, 30, 60, 60, 60, 60, 120, 120, 120)
21
22 # Costs
23 setup_cost <- c(15000, 15000, 15000, 20000, 20000, 20000, 20000, 25000, 25000, 25000)
24 allocation_cost <- c(180, 180, 180, 200, 200, 200, 200, 220, 220, 220)
25
26 # Weekly demand
27 P_diag <- c(230, 235, 225, 230, 240, 245, 280, 295, 300, 285, 240, 235, 230, 225, 230, 235, 240, 245, 250,
28             245, 240, 235, 230, 230, 235, 230) # Diagnostic Hours, 26 weeks
29 P_thera <- c(145, 145, 150, 150, 155, 160, 168, 176, 182, 188, 190, 192, 188, 195, 187, 182, 178,
30             172, 168, 165, 162, 158, 155, 152, 150, 148) # Therapeutic Hours, 26 weeks
31 C_w <- c(520, 515, 525, 520, 530, 525, 490, 480, 485, 500, 510, 515, 520, 525, 530, 525, 520,
32          480, 470, 475, 500, 510, 515, 520, 525, 530) # clinician availability, 26 weeks
33 weeks <- 1:26
34
35 ## building model
36 model <- MIPModel() %>%
37   # Decision variables
38   add_variable(x_diag[r,w], r = room, w = weeks, type = "binary") %>%
39   # ...
```

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help

data x r studio code assignment.R* x R code d3m task 1.R* x mutiple linear regression.R x linear regression.R* x
Source on Save Run Source

34
35 ## building model
36 model <- MIPModel() %>%
37   # Decision variables
38   add_variable(x_diag[r,w], r = room, w = weeks, type = "binary") %>%
39   add_variable(x_thera[r,w], r = room, w = weeks, type = "binary") %>%
40   add_variable(d[r,w], r = room, w = weeks, type = "continuous", lb = 0) %>%
41   add_variable(t[r,w], r = room, w = weeks, type = "continuous", lb = 0) %>%
42   add_variable(s[r,w], r = room, w = weeks, type = "binary")
43
44 ## room configuration
45 model <- model %>%
46   add_constraint(x_diag[r,w] + x_thera[r,w] <= 1, r = room, w = weeks)
47
48 ## room capacity
49 model <- model %>%
50   add_constraint(d[r, w] <= diag_capacity[r] * x_diag[r, w], r = room, w = weeks) %>%
51   add_constraint(t[r, w] <= thera_capacity[r] * x_thera[r, w], r = room, w = weeks)
52
53 ## Demand satisfaction
54 model <- model %>%
55   add_constraint(sum_expr(d[r,w], r = room) >= P_diag[w], w = weeks) %>%
56   add_constraint(sum_expr(t[r,w], r = room) >= P_thera[w], w = weeks)
57
58 ## Clinical availability
59 model <- model %>%
60   add_constraint(sum_expr(d[r,w] + t[r,w], r = room) <= C_w[w], w = weeks)
61
62 # Setup logic
63
64 # Week 1 (no previous week exists)
65 model <- model %>%
66   add_constraint(
67     s[r,1] >= x_diag[r,1] + x_thera[r,1],
68     r = room
69   )
70
71 # Weeks 2 to 26
72
4:4 (Top Level) R Script
Console

```

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins

data x r studio code assignment.R x R code d3m task 1.R x mutiple linear regression.R x linear regression.R x

Source on Save Run Source

```
67     s[r,1] >= x_diag[r,1] + x_thera[r,1],
68     r = room
69   )
70
71   # Weeks 2 to 26
72   model <- model %>%
73     add_constraint(
74       s[r,w] >= x_diag[r,w] + x_thera[r,w] -
75         (x_diag[r,w-1] + x_thera[r,w-1]),
76       r = room, w = 2:26
77     )
78
79   ## Objective function
80   model <- model %>%
81     set_objective(sum_expr(setup_cost[r]*s[r,w] + allocation_cost[r]*(d[r,w]+t[r,w])),
82       r = room, w = weeks), sense = "min")
83
84   ## Solve milp
85   # Solve model (with time limit)
86   # Solve MILP with 10-minute time limit (600,000 ms)
87   result <- solve_model(
88     model,
89     with_ROI(
90       solver = "glpk",
91       verbose = TRUE,
92       control = list(tm_limit = 600000) # 10 minutes in milliseconds
93     )
94   )
95
96   # Check solver status
97   solver_status(result)
98
99   # Get objective value and variable values
100  objective_value(result)
101  get_solution(result, d[r, w])
102  get_solution(result, t[r, w])
103  get_solution(result, x_diag[r, w])
104  get_solution(result, x_thera[r, w])
105
```

4:4 (Top Level) R Script

Console

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
+ - Go to file/function Addins
data x r studio code assignment.R* x R code d3m task 1.R* x mutiple linear regression.R x linear regression.R* x
Source on Save Run Source
103 get_solution(result, x_diag[r, w])
104 get_solution(result, x_thera[r, w])
105
106 ## sanity check
107 sum(P_diag)
108 sum(P_thera)
109
110 ## compare with results
111 summary(get_solution(result, d[r,w])$value)
112
113 ## visualization
114 library(ggplot2)
115
116 total_diag <- get_solution(result, d[r,w]) %>% group_by(w) %>% summarise(total = sum(value))
117 ggplot(total_diag, aes(x=w, y=total)) +
118   geom_bar(stat="identity") +
119   labs(title="Total Diagnostic Hours per Week")
120
121 total_thera <- get_solution(result, t[r,w]) %>%
122   group_by(w) %>%
123   summarise(total = sum(value))
124
125 ggplot(total_thera, aes(x=w, y=total)) +
126   geom_bar(stat="identity") +
127   labs(title="Total Therapeutic Hours per Week")
128
129 room_usage <- get_solution(result, d[r,w]) %>%
130   group_by(r) %>%
131   summarise(total = sum(value))
132
133 ggplot(room_usage, aes(x=factor(r), y=total)) +
134   geom_bar(stat="identity") +
135   labs(title="Total Diagnostic Hours by Room",
136        x="Room",
137        y="Total Hours")
138
139
140
4:4 (Top Level) R Script
Console
```

```
RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

+ - [Icons] Go to file/function [Icons] Addins

Source

Console Terminal Background Jobs

R - R 4.5.1 - D:/Sanjai/D3M/

+1124423: mip = 2.197880000e+06 >= 2.173266667e+06 1.1% (167192; 22550)
> ## Solve milp
> # Solve model (with time limit)
> # Solve MILP with 10-minute time limit (600,000 ms)
> result <- solve_model(
+ model,
+ with_ROI(
+ solver = "glpk",
+ verbose = TRUE,
+ control = list(tm_limit = 600000) # 10 minutes in milliseconds
+ )
+ )
<SOLVER MSG> ----
GLPK Simplex Optimizer 5.0
1118 rows, 1300 columns, 3880 non-zeros
0: obj = 0.000000000e+00 inf = 1.070e+04 (52)
Perturbing LP to avoid stalling [199]...
680: obj = 2.435500000e+06 inf = 3.886e-16 (0)
Removing LP perturbation [1243]...
* 1243: obj = 2.170574167e+06 inf = 1.019e-13 (0) 4
OPTIMAL LP SOLUTION FOUND
GLPK Integer Optimizer 5.0
1118 rows, 1300 columns, 3880 non-zeros
780 integer variables, all of which are binary
Integer optimization begins...
Long-step dual simplex will be used
+ 1243: mip = not found yet >= -inf (1; 0)
+ 2576: >>>> 2.299020000e+06 >= 2.170774167e+06 5.6% (237; 3)
+ 3204: >>>> 2.246300000e+06 >= 2.170774167e+06 3.4% (354; 38)
+ 13107: >>>> 2.228600000e+06 >= 2.171361667e+06 2.6% (2049; 105)
+ 14401: >>>> 2.228340000e+06 >= 2.171366667e+06 2.6% (2165; 365)
+ 18755: >>>> 2.223180000e+06 >= 2.171489167e+06 2.3% (2899; 477)
+ 27060: mip = 2.223180000e+06 >= 2.171594167e+06 2.3% (4462; 816)
+ 36785: mip = 2.223180000e+06 >= 2.171894167e+06 2.3% (6087; 851)
+ 42921: >>>> 2.221540000e+06 >= 2.171894167e+06 2.2% (7472; 863)
+ 43433: >>>> 2.201440000e+06 >= 2.171894167e+06 1.3% (7130; 1783)
+ 45880: >>>> 2.199640000e+06 >= 2.171894167e+06 1.3% (6606; 3904)
+ 54421: mip = 2.199640000e+06 >= 2.171907500e+06 1.3% (7643; 4902)
+ 63884: mip = 2.199640000e+06 >= 2.171944167e+06 1.3% (9181; 4924)
+ 71936: mip = 2.199640000e+06 >= 2.171974167e+06 1.3% (10916; 4945)
```

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins

Source

Console Terminal Background Jobs

R 4.5.1 · D:/Sanjai/D3M/

```

+ 34421: mip = 2.199640000e+06 >= 2.171907500e+06 1.3% (7043; 4902)
+ 63884: mip = 2.199640000e+06 >= 2.171944167e+06 1.3% (9181; 4924)
+ 71936: mip = 2.199640000e+06 >= 2.171974167e+06 1.3% (10916; 4945)
+ 81378: mip = 2.199640000e+06 >= 2.172066667e+06 1.3% (12366; 4968)
+ 90555: mip = 2.199640000e+06 >= 2.172161667e+06 1.2% (13724; 4995)
+ 98919: mip = 2.199640000e+06 >= 2.172251667e+06 1.2% (15353; 5017)
+106604: mip = 2.199640000e+06 >= 2.172256667e+06 1.2% (16848; 5040)
Time used: 60.0 secs. Memory used: 378.1 Mb.
+115098: mip = 2.199640000e+06 >= 2.172259167e+06 1.2% (18394; 5062)
+123915: mip = 2.199640000e+06 >= 2.172266667e+06 1.2% (19913; 5088)
+133182: mip = 2.199640000e+06 >= 2.172277500e+06 1.2% (21390; 5114)
+141817: mip = 2.199640000e+06 >= 2.172316667e+06 1.2% (22900; 5137)
+150250: mip = 2.199640000e+06 >= 2.172316667e+06 1.2% (24346; 5162)
+159623: mip = 2.199640000e+06 >= 2.172366667e+06 1.2% (25911; 5188)
+168364: mip = 2.199640000e+06 >= 2.172416667e+06 1.2% (27456; 5214)
+176619: mip = 2.199640000e+06 >= 2.172466667e+06 1.2% (29041; 5240)
+183454: >>>> 2.197880000e+06 >= 2.172616667e+06 1.1% (30205; 5263)
+191058: mip = 2.197880000e+06 >= 2.172666667e+06 1.1% (26088; 15945)
+200470: mip = 2.197880000e+06 >= 2.172691667e+06 1.1% (27479; 15980)
+213359: mip = 2.197880000e+06 >= 2.172691667e+06 1.1% (28647; 16018)
Time used: 120.0 secs. Memory used: 388.3 Mb.
+223892: mip = 2.197880000e+06 >= 2.172711667e+06 1.1% (29942; 16052)
+232116: mip = 2.197880000e+06 >= 2.172736667e+06 1.1% (31172; 16112)
+241600: mip = 2.197880000e+06 >= 2.172736667e+06 1.1% (32417; 16168)
+250559: mip = 2.197880000e+06 >= 2.172736667e+06 1.1% (33684; 16224)
+259831: mip = 2.197880000e+06 >= 2.172736667e+06 1.1% (34888; 16276)
+268099: mip = 2.197880000e+06 >= 2.172739167e+06 1.1% (36390; 16308)
+276914: mip = 2.197880000e+06 >= 2.172746667e+06 1.1% (37876; 16341)
+287881: mip = 2.197880000e+06 >= 2.172756667e+06 1.1% (39102; 16386)
+295026: mip = 2.197880000e+06 >= 2.172786667e+06 1.1% (40309; 16437)
+301935: mip = 2.197880000e+06 >= 2.172786667e+06 1.1% (41424; 16503)
+308484: mip = 2.197880000e+06 >= 2.172786667e+06 1.1% (42502; 16573)
+316248: mip = 2.197880000e+06 >= 2.172786667e+06 1.1% (43704; 16639)
Time used: 180.0 secs. Memory used: 404.0 Mb.
+322839: mip = 2.197880000e+06 >= 2.172789167e+06 1.1% (44835; 16688)
Time used: 279.7 secs. Memory used: 404.8 Mb.
+325439: mip = 2.197880000e+06 >= 2.172789167e+06 1.1% (45280; 16698)
+333060: mip = 2.197880000e+06 >= 2.172794167e+06 1.1% (46207; 16754)
+340782: mip = 2.197880000e+06 >= 2.172816667e+06 1.1% (47540; 16794)
+348161: mip = 2.197880000e+06 >= 2.172866667e+06 1.1% (48697; 16848)
+357103: mip = 2.197880000e+06 >= 2.172866667e+06 1.1% (50036; 16883)

```

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins

Source

Console Terminal x Background Jobs x

R 4.5.1 · D:/Sanjai/D3M/ ↗

```
+316248: mip = 2.197880000e+06 >= 2.172786667e+06 1.1% (43704; 16639)
Time used: 180.0 secs. Memory used: 404.0 Mb.
+322839: mip = 2.197880000e+06 >= 2.172789167e+06 1.1% (44835; 16688)
Time used: 279.7 secs. Memory used: 404.8 Mb.
+325439: mip = 2.197880000e+06 >= 2.172789167e+06 1.1% (45280; 16698)
+333060: mip = 2.197880000e+06 >= 2.172794167e+06 1.1% (46207; 16754)
+340782: mip = 2.197880000e+06 >= 2.172816667e+06 1.1% (47540; 16794)
+348161: mip = 2.197880000e+06 >= 2.172866667e+06 1.1% (48697; 16848)
+357103: mip = 2.197880000e+06 >= 2.172866667e+06 1.1% (50036; 16883)
+367869: mip = 2.197880000e+06 >= 2.172866667e+06 1.1% (51294; 16921)
+376764: mip = 2.197880000e+06 >= 2.172866667e+06 1.1% (52672; 16954)
+386068: mip = 2.197880000e+06 >= 2.172866667e+06 1.1% (53843; 16992)
+393573: mip = 2.197880000e+06 >= 2.172886667e+06 1.1% (55048; 17038)
+402718: mip = 2.197880000e+06 >= 2.172896667e+06 1.1% (56164; 17083)
+411737: mip = 2.197880000e+06 >= 2.172916667e+06 1.1% (57485; 17116)
Time used: 339.7 secs. Memory used: 416.7 Mb.
+420082: mip = 2.197880000e+06 >= 2.172916667e+06 1.1% (58843; 17152)
+428029: mip = 2.197880000e+06 >= 2.172916667e+06 1.1% (60247; 17185)
+436749: mip = 2.197880000e+06 >= 2.172924167e+06 1.1% (61345; 17238)
+444982: mip = 2.197880000e+06 >= 2.172957500e+06 1.1% (62505; 17281)
+452316: mip = 2.197880000e+06 >= 2.173006667e+06 1.1% (63788; 17323)
+458306: mip = 2.197880000e+06 >= 2.173006667e+06 1.1% (65346; 17356)
+465405: mip = 2.197880000e+06 >= 2.173009167e+06 1.1% (66764; 17387)
+474027: mip = 2.197880000e+06 >= 2.173011667e+06 1.1% (67719; 17451)
+482624: mip = 2.197880000e+06 >= 2.173011667e+06 1.1% (68584; 17519)
+490240: mip = 2.197880000e+06 >= 2.173016667e+06 1.1% (69747; 17559)
+498101: mip = 2.197880000e+06 >= 2.173024167e+06 1.1% (71109; 17581)
+507558: mip = 2.197880000e+06 >= 2.173026667e+06 1.1% (72220; 17627)
Time used: 399.7 secs. Memory used: 429.0 Mb.
+517099: mip = 2.197880000e+06 >= 2.173036667e+06 1.1% (73296; 17673)
+524731: mip = 2.197880000e+06 >= 2.173061667e+06 1.1% (74471; 17711)
+532928: mip = 2.197880000e+06 >= 2.173061667e+06 1.1% (75393; 17773)
+541512: mip = 2.197880000e+06 >= 2.173066667e+06 1.1% (76484; 17819)
+550197: mip = 2.197880000e+06 >= 2.173074167e+06 1.1% (77710; 17843)
+557602: mip = 2.197880000e+06 >= 2.173094167e+06 1.1% (78726; 17898)
+566310: mip = 2.197880000e+06 >= 2.173094167e+06 1.1% (79737; 17941)
+575165: mip = 2.197880000e+06 >= 2.173094167e+06 1.1% (80710; 17992)
+584821: mip = 2.197880000e+06 >= 2.173107500e+06 1.1% (81833; 18030)
+593078: mip = 2.197880000e+06 >= 2.173107500e+06 1.1% (83125; 18050)
+600721: mip = 2.197880000e+06 >= 2.173107500e+06 1.1% (84525; 18068)
```

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins

Source

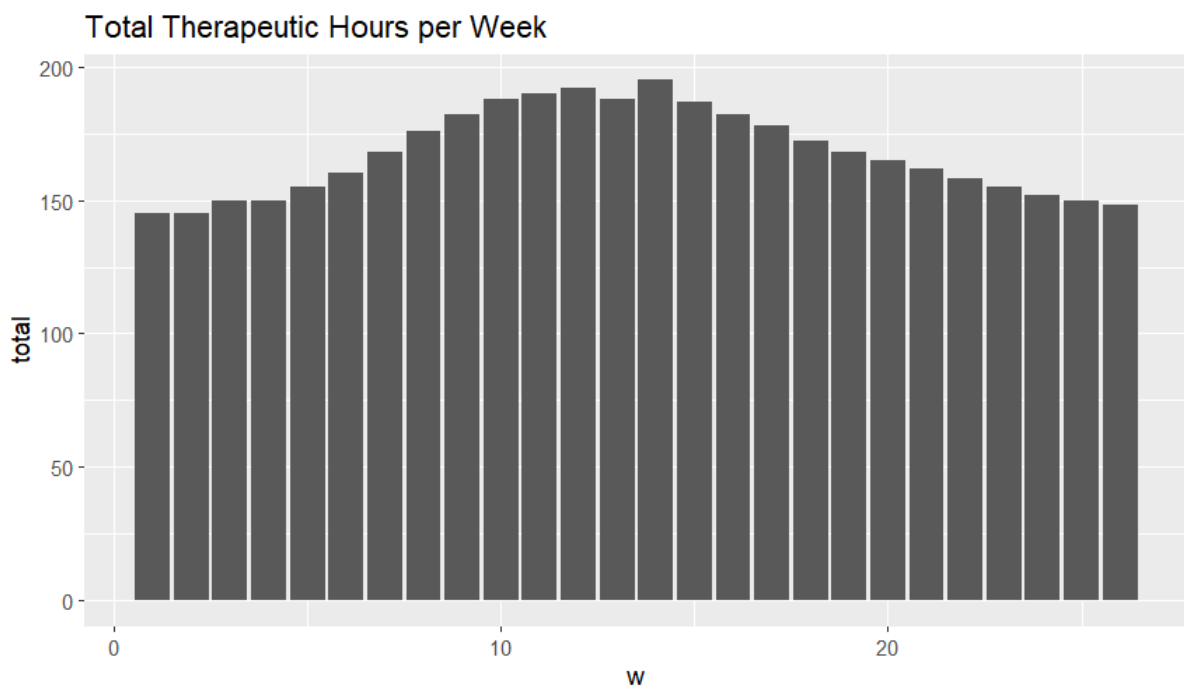
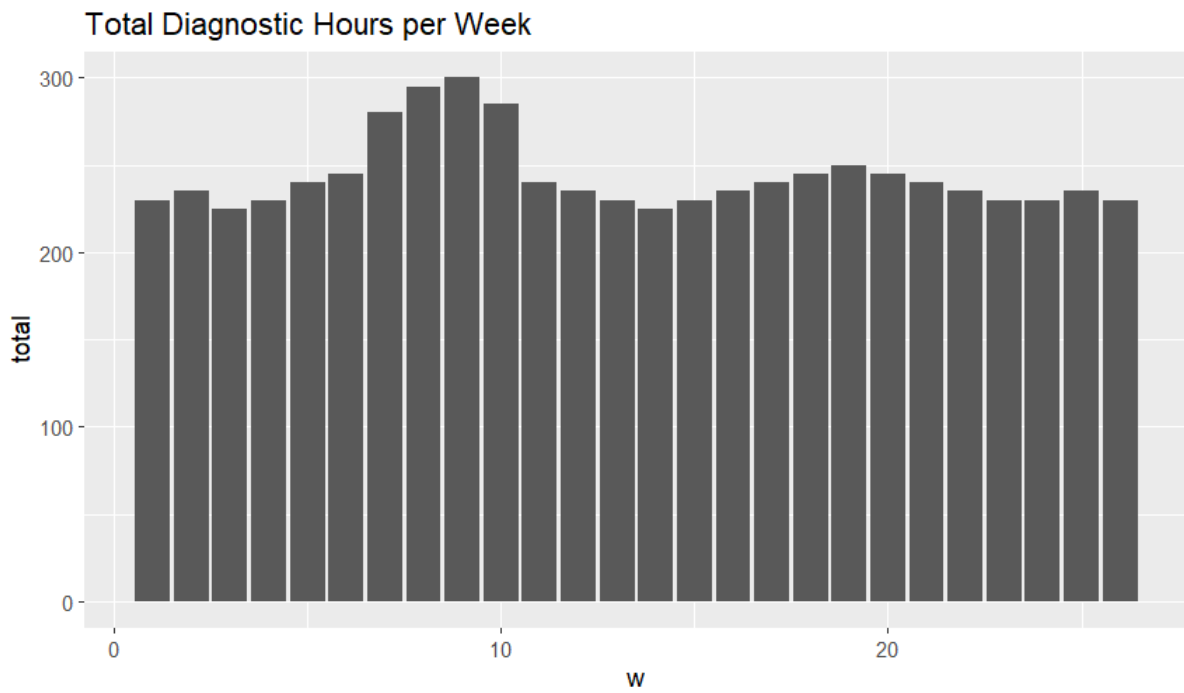
Console Terminal Background Jobs

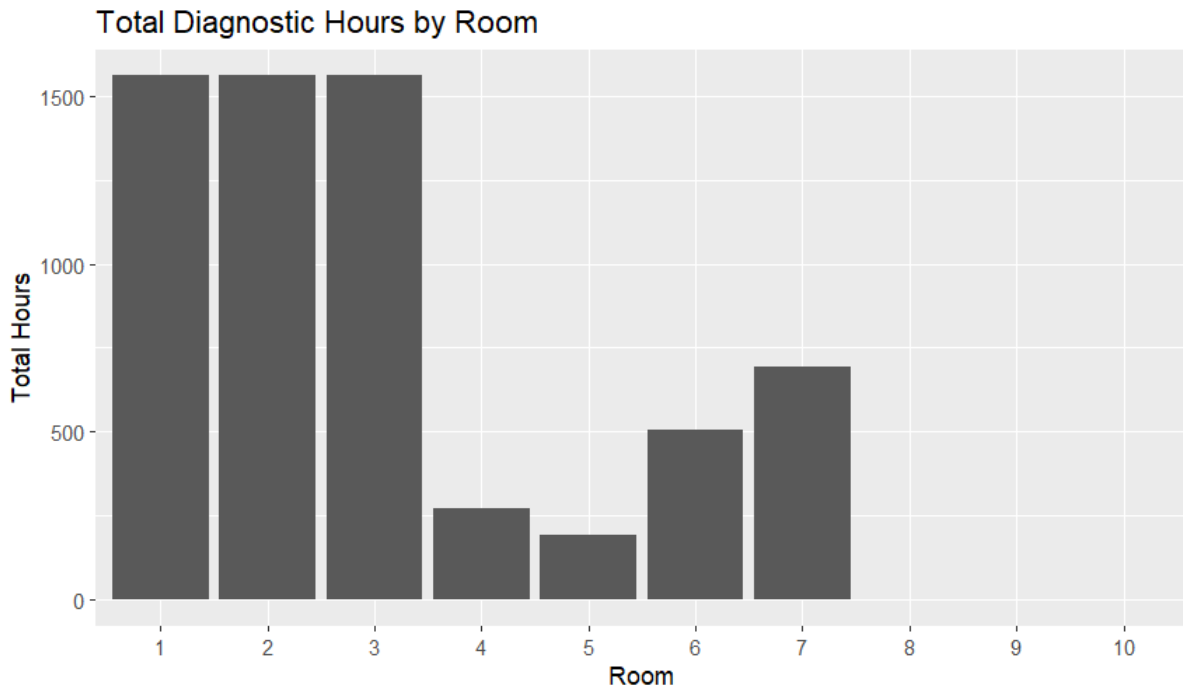
```

R 4.5.1 · D:/Sanjai/D3M/
+490240: mip = 2.197880000e+06 >= 2.173010007e+06 1.1% (69747, 17339)
+498101: mip = 2.197880000e+06 >= 2.173024167e+06 1.1% (71109; 17581)
+507558: mip = 2.197880000e+06 >= 2.173026667e+06 1.1% (72220; 17627)
Time used: 399.7 secs. Memory used: 429.0 Mb.
+517099: mip = 2.197880000e+06 >= 2.173036667e+06 1.1% (73296; 17673)
+524731: mip = 2.197880000e+06 >= 2.173061667e+06 1.1% (74471; 17711)
+532928: mip = 2.197880000e+06 >= 2.173061667e+06 1.1% (75393; 17773)
+541512: mip = 2.197880000e+06 >= 2.173066667e+06 1.1% (76484; 17819)
+550197: mip = 2.197880000e+06 >= 2.173074167e+06 1.1% (77710; 17843)
+557602: mip = 2.197880000e+06 >= 2.173094167e+06 1.1% (78726; 17898)
+566310: mip = 2.197880000e+06 >= 2.173094167e+06 1.1% (79737; 17941)
+575165: mip = 2.197880000e+06 >= 2.173094167e+06 1.1% (80710; 17992)
+584821: mip = 2.197880000e+06 >= 2.173107500e+06 1.1% (81833; 18030)
+593078: mip = 2.197880000e+06 >= 2.173107500e+06 1.1% (83125; 18050)
+600721: mip = 2.197880000e+06 >= 2.173107500e+06 1.1% (84525; 18068)
+607502: mip = 2.197880000e+06 >= 2.173116667e+06 1.1% (85654; 18116)
Time used: 459.7 secs. Memory used: 443.5 Mb.
+615550: mip = 2.197880000e+06 >= 2.173116667e+06 1.1% (86843; 18163)
+622238: mip = 2.197880000e+06 >= 2.173116667e+06 1.1% (87959; 18216)
+629188: mip = 2.197880000e+06 >= 2.173116667e+06 1.1% (89165; 18251)
+636495: mip = 2.197880000e+06 >= 2.173116667e+06 1.1% (90221; 18294)
+644300: mip = 2.197880000e+06 >= 2.173116667e+06 1.1% (91290; 18336)
+648779: mip = 2.197880000e+06 >= 2.173134444e+06 1.1% (91870; 18360)
Time used: 550.5 secs. Memory used: 448.1 Mb.
+652156: mip = 2.197880000e+06 >= 2.173136667e+06 1.1% (92339; 18386)
+660164: mip = 2.197880000e+06 >= 2.173144167e+06 1.1% (93229; 18433)
+665634: mip = 2.197880000e+06 >= 2.173145556e+06 1.1% (93971; 18512)
+672900: mip = 2.197880000e+06 >= 2.173157500e+06 1.1% (95192; 18545)
+679332: mip = 2.197880000e+06 >= 2.173166667e+06 1.1% (96060; 18607)
+685124: mip = 2.197880000e+06 >= 2.173166667e+06 1.1% (97208; 18653)
+690144: mip = 2.197880000e+06 >= 2.173166667e+06 1.1% (98289; 18692)
+698907: mip = 2.197880000e+06 >= 2.173186667e+06 1.1% (98998; 18752)
+706663: mip = 2.197880000e+06 >= 2.173206667e+06 1.1% (100038; 18786)
+710153: mip = 2.197880000e+06 >= 2.173206667e+06 1.1% (100756; 18839)
TIME LIMIT EXCEEDED; SEARCH TERMINATED
<!SOLVER MSG> ----

```

Visualization plots





7. Decision and Interpretation

The optimisation model determines the optimal configuration of procedure rooms and allocation of diagnostic and therapeutic hours for each week of the planning horizon.

The resulting solution satisfies all demand, capacity, and clinician availability constraints while minimising the total operational and setup costs. The optimal solution identifies which rooms should operate each week and how many diagnostic and therapeutic hours should be allocated while minimising total cost.

8. Practical Implications

The optimisation model provides clear insights into how the endoscopy unit can allocate its resources efficiently over the 26-week planning horizon. From the results, it is evident that **small rooms are heavily utilised**, while **large rooms remain underused**. This occurs because the model seeks to minimise total costs: small rooms have lower setup and operating costs, and the demand for procedures can often be met without activating the larger, more expensive rooms.

From a managerial perspective, this has several implications:

1. **Cost Efficiency vs. Room Utilisation:** The heavy use of small rooms ensures lower operational costs, but it may lead to scheduling pressure on these rooms, potentially

creating bottlenecks during peak weeks. Managers should monitor utilisation closely to avoid overworking staff or causing delays.

2. **Large Room Underutilisation:** Large rooms remain underused because the model only activates them when smaller rooms cannot meet procedure demand. While this minimises cost, it may lead to idle capacity and inefficient use of the hospital's fixed assets. Managers could consider alternative strategies, such as grouping complex procedures that require larger rooms or flexible scheduling across weeks to balance usage.
3. **Impact on Staff Workload:** Concentrating procedures in small rooms can lead to uneven distribution of workload among clinicians. Staff assigned to small rooms may experience higher pressure, whereas clinicians in large rooms may have idle time. Managers should consider cross-training staff or rotating assignments to maintain fairness and efficiency.
4. **Operational Recommendations:**
 - Review procedure scheduling weekly to anticipate peak demands and decide whether activating large rooms is justified despite higher costs.
 - Implement staggered appointment slots to reduce bottlenecks in small rooms.
 - Explore using large rooms for combined diagnostic and therapeutic procedures to improve utilisation.
 - Monitor overtime and idle times for clinicians to maintain staff satisfaction and patient care quality.

In summary, the model gives a **cost-optimised schedule**, but managers must balance **financial efficiency with practical resource utilisation and staff workload**. These insights allow the hospital to make data-driven operational decisions, ensuring high-quality patient care while controlling costs.

AI Declaration:

I declare that I wrote this assignment myself. I used AI to help improve the quality and clarity of the R code and suggest minor improvements, but all implementation and solutions are my own work.

TASK 2

MILP Application in a Healthcare Setting

Healthcare Problem

The article by **Klaas et al. (2024)** investigates this issue within a nephrology outpatient clinic in a Dutch hospital. Patients with chronic kidney disease (CKD) require long-term monitoring and follow-up consultations with nephrologists. Traditionally, these follow-up appointments are scheduled months in advance as part of a standardised care pathway, regardless of the patient's current health condition. As a result, many patients attend consultations even when their clinical status has not changed.

.This situation creates operational inefficiencies. Patients who do not require immediate clinical attention occupy appointment slots that could otherwise be used for new patients or those experiencing disease progression. At the same time, hospitals face increasing patient volumes and shortages of healthcare professionals. Efficient utilisation of outpatient clinic capacity has therefore become a critical operational challenge.

To address this problem, the study proposes a strategy that combines **patient pre-assessment with mathematical optimisation of appointment scheduling**. Pre-assessment evaluates whether a patient actually requires a consultation before scheduling an appointment. However, this approach introduces variability in patient demand, making scheduling more complex. To manage this variability and maintain operational efficiency, the authors develop an optimisation-based scheduling model.

The purpose of the study is therefore to design and evaluate a method that improves outpatient clinic efficiency by identifying unnecessary appointments and optimally allocating available appointment slots. By integrating clinical decision criteria with operations research methods, the study aims to improve access to care while ensuring efficient use of limited healthcare resources.

MILP Model Outline

The optimisation model determines an efficient scheduling strategy for outpatient appointments after patient pre-assessment.. The model aims to allocate appointment slots in a way that balances patient demand with available clinic capacity while minimising inefficiencies such as idle time and overtime.

The key components of the optimisation model can be summarised as follows.

Decision Variables

The model determines how many appointment slots should be scheduled for different patient types in each week. The decision variables therefore represent the allocation of outpatient appointment slots within the clinic schedule.

Objective Function

The objective of the model is to minimise operational inefficiencies in the outpatient clinic. Specifically, the model seeks to minimise:

- the expected number of unused appointment slots (idle time),
- the probability that staff must work overtime, and
- the total amount of overtime when it occurs.

These objectives reflect the operational goals of healthcare managers, who aim to maximise resource utilisation while avoiding excessive workloads for clinical staff.

Constraints

Several operational constraints ensure that the scheduling solution remains feasible and realistic.

Capacity constraints ensure that the number of planned appointments does not exceed the available clinic capacity. The nephrology clinic in the study operates with a fixed number of appointment slots per week.

Demand constraints reflect the variability in patient demand. Since pre-assessment determines whether patients require appointments on short notice, the exact number of required consultations varies across weeks.

Overtime constraints limit the probability that the clinic schedule will exceed the planned capacity. In the study, the allowed probability of overtime is set to 10%.

Operational constraints also ensure that appointment slots are allocated according to the available workforce capacity of nephrologists.

The model uses historical patient data to estimate demand distributions and determine an optimal “blueprint schedule” that specifies the number of appointment slots that should be planned each week.

Exact Solution Method

The optimisation model in the study is solved using a **stochastic programming approach**, which is an exact mathematical optimisation technique commonly used in operations research.

Stochastic programming is particularly suitable for healthcare scheduling problems because patient demand is uncertain. Instead of assuming a fixed demand level, stochastic programming considers multiple possible demand scenarios and identifies solutions that perform well across all scenarios.

In this study, the stochastic programming model generates different possible patient demand scenarios using historical outpatient clinic data. The optimisation algorithm then evaluates potential scheduling strategies and identifies the one that minimises expected operational inefficiencies.

The model produces a **blueprint schedule**, which specifies the optimal number of appointment slots that should be reserved for different types of consultations. This schedule serves as a planning guideline for clinic managers.

After determining the optimal scheduling strategy, the authors evaluate its performance using **Monte Carlo simulation**. Simulation allows the researchers to test how the scheduling strategy performs under thousands of possible demand scenarios. This provides a realistic estimate of expected idle time, overtime, and clinic utilisation.

The combination of stochastic programming and simulation enables the researchers to evaluate scheduling policies before implementing them in clinical practice. This reduces operational risks and helps healthcare managers make data-driven planning decisions.

Evaluation of the Method

The optimisation approach used in the study offers several important advantages for healthcare operations management.

First, Stochastic programming is suitable for healthcare scheduling because patient demand is uncertain. The method evaluates multiple demand scenarios and identifies solutions that perform well across these possible outcomes. the model produces scheduling solutions that remain robust under uncertain conditions.

Second, the method supports efficient resource utilisation. By optimally allocating appointment slots, the clinic can reduce unused capacity while also limiting excessive overtime. This improves staff workload management and increases access to care for patients who require consultation.

However, the method also has some limitations. One limitation is that optimisation models rely on assumptions about patient demand distributions and operational parameters. If these assumptions do not accurately reflect real-world conditions, the resulting scheduling strategies may perform less effectively.

Another limitation is computational complexity. Stochastic programming models can become computationally intensive when the number of scenarios or decision variables increases. However, modern optimisation solvers and computing resources have significantly improved the practicality of these methods.

Despite these limitations, the method is well suited for the healthcare scheduling problem studied in the article, as it balances operational efficiency with uncertainty in patient demand.

Practical Insights and Limitations

The results of the study provide several practical insights for healthcare operations management.

The retrospective analysis of outpatient clinic data revealed that approximately **18% of scheduled appointments were unnecessary** according to the pre-assessment criteria. This indicates a significant opportunity to improve clinic efficiency by reducing routine follow-up consultations that are not medically required.

After implementing the optimisation-based scheduling strategy, the clinic was able to reduce planned appointment capacity by **13%** while still meeting patient demand. This freed up clinical resources that could potentially be used to treat additional patients.

The study also shows that the combined pre-assessment and optimisation approach allows outpatient clinics to manage **up to a 20% increase in patient demand** without requiring additional capacity. This finding is particularly important given the increasing pressure on healthcare systems worldwide.

Despite these benefits, several limitations must be considered. The pre-assessment criteria used in the study were based on clinical consensus and may not fully capture individual patient needs. Some patients who technically meet the criteria for an appointment may still benefit from clinical consultation.

Additionally, the study focuses on a single nephrology outpatient clinic in the Netherlands. While the modelling approach is generalisable, the specific results may vary in other healthcare settings due to differences in patient populations, clinical practices, and resource availability.

Finally, patient preferences and communication with clinicians play an important role in outpatient care. Future research should incorporate patient perspectives and personalised care pathways into optimisation models.

Overall, the study demonstrates how operations research methods such as stochastic programming can support more efficient healthcare delivery while maintaining high standards of patient care.

References (Harvard Style)

Klaas, R., Lok-Visser, J., Doornebal, J., Roelofs, T., Rachuba, S. and Leefink, G. (2024). Criteria-based outpatient scheduling at a nephrology clinic: prospective evaluation of patient pre-assessment and its corresponding adaptive scheduling strategy. *BMC Health Services Research*, 24, 1145. <https://doi.org/10.1186/s12913-024-11615-7>